

Reading Cache Polynomials as Locality Regimes

A study of how to use the output of algebraic loop locality analysis

This directory contains a self-contained analysis built on the algebraic locality compiler (the “imaginary reuses” paper: symbolic reuse-interval distributions from parametric polytopes, converted by Denning Recursion into cache-size and miss-ratio polynomials). Everything here is derived fresh from raw analyzer output, under the paper’s canonical configuration: **infinite repeat, scale approximation in Barvinok, block size 8** (one 64-byte line of eight f64 elements). The one exception is an element-granularity run of naive matrix multiplication used to reproduce the paper’s Table 1 exactly.

The question addressed is not whether the model is accurate — the paper establishes that — but *how its output should be read and what it is for*. The previous attempt in this repository read the output through data-movement complexity: collapse the symbolic distribution into one scalar aggregate, keep its leading term, and classify kernels by growth exponents. Section 2 shows why that reading is structurally unsound. Sections 3–6 develop and evaluate an alternative: read the cache polynomials as a finite list of **regimes** — (cache-size boundary, miss-ratio plateau) pairs — that is, as an exact two-parameter function $mr(C, n)$, and treat every practical question as a functional of that object. Sections 7 and 8 are the two side studies requested: co-scaling / scaling laws, and a first-order extrapolation to parallel execution.

Summary of what is demonstrated, all computed from analyzer output alone:

1. Scalar data-movement aggregates misrank kernels at real cache sizes (an order inversion and several 4–6x near-ties are exhibited), and the reason is structural: RI Sum Invariance forces value-weighted aggregates to be dominated by exactly the levels whose probability vanishes (Section 2).
2. The regime list is small (4–18 displayed levels per kernel), exact, and reproduces the paper’s published tables (Section 4).
3. It answers, in closed form: what cache a problem needs, what problem a cache fits, what a transformation buys at each cache size, and how cache must co-scale with data (Sections 3, 5, 6, 7).
4. Because the polynomials are symbolic in *every* loop bound, parallel slicing is parameter substitution: per-worker miss curves need no new analysis. This predicts which decompositions let p caches act as one (j/k -slicing of gemm; threshold p^* matches the boundary formula) and which cannot (i -slicing; aggregate traffic provably flat in p), plus a line-granularity floor invisible to element-level reasoning (Section 8).
5. The framework carries its own consistency checks — mass conservation and RI Sum Invariance — which caught real inconsistencies in analyzer output for the time-stepped stencils, and localized a filtered imaginary level in gemm (Section 4.3).

1. The object under study

For one kernel, the analyzer emits piecewise quasi-polynomials in all loop-bound parameters ($p_0, p_1, \dots$):

- the total access count A ;
- the **RI distribution**: pairs (ri_j, w_j) — a reuse-interval value and the number of accesses realizing it;
- the **RD distribution**: pairs (rd_j, w_j) — the number of distinct cache blocks touched inside the reuse window (the per-reuse LRU stack distance, computed under the scale approximation) and its access count.

Under **infinite repeat** every access is a reuse, real or imaginary; a fully associative LRU cache of C blocks hits a reuse iff $rd \leq C$, so

$$mr(C, n) = (\text{sum of } w_j \text{ with } rd_j > C) / A.$$

This staircase is the entire content of the model. The processing done here (`qp.py`, `regimes.py`, `derived.py`) is:

1. **Bind** all parameters to one size n (convolution is the exception: its filter is a second scale and is bound separately; see 4.3).
2. **Fit** every rd_j and w_j exactly as a polynomial in n by sampling on a fixed residue class (multiples of 8400) and solving exact Vandermonde systems with held-out verification points. Fractions throughout; no floating point in any symbolic statement.
3. **Cluster** rd entries into **levels**: two entries belong to the same level iff their rd polynomials have the same degree and the same leading coefficient. A level is a physical scale of the loop nest (a row, a matrix, the whole footprint), not an artifact of chamber decomposition.
4. For each level k , compute exactly: the boundary $c_k(n)$ (largest rd in the level; the mass-weighted average is also kept), the portion $p_k(n) = \text{mass}/A$, and the plateau $m_k(n) = \text{tail mass above level } k \text{ over } A$ — the miss ratio of a cache that holds levels $1..k$.
5. Concrete numbers at a specific n are evaluated from the raw piecewise quasi-polynomials with their guards (never from fits off the sampling ray), gated by each entry's region domain via a small Fourier-Motzkin feasibility check. This matters: piece guards are printed gisted against the region domains, so ungated evaluation over-counts.

A level whose portion does not vanish as n grows, or whose plateau changes the leading behavior of the miss ratio, is **displayed**; the rest are folded in (they still participate in exact evaluation).

Terminology. For a kernel with data footprint $D(n)$ blocks, the displayed levels give the *regime diagram*: cache sizes in $[c_k, c_{k+1})$ form regime k with miss ratio $m_k(n)$. Boundaries are polynomials (possibly with fractional leading coefficients), plateaus are rational functions of n .

2. Why the leading-term reading fails

Naive matrix multiplication (three accesses per iteration, element granularity, infinite repeat) has the RI table (paper Table 1; reproduced exactly by this pipeline, Section 4.1):

RI	$P(ri)$	$c(ri)$	$m(ri)$ after
3	$1/3 - 1/(3n)$	3	$2/3 + 1/(3n)$
$3n$	$1/3 - 1/(3n)$	$2n + 2 - 1/n$	$1/3 + 2/(3n)$
$3n^2$	$1/3$	$n^2 + 3n - 1/n$	$2/(3n)$
$\sim 3n^3$ (imaginary)	$2/(3n)$	$\sim 3n^2$	0

The paradox that motivated this study is visible in the last row: the largest reuse intervals carry *vanishing* probability, so their direct contribution to any miss ratio is asymptotically nil — yet any “complexity”-style scalar is dominated by them. This is not an accident of matmul; it is forced by **RI Sum Invariance** (paper, Section 2.5):

$$\sum_k ri_k \cdot P(ri_k) = D \quad (\text{the data size}).$$

The products value $\times$ portion are bounded by the same invariant for every kernel; the top levels always carry a constant share of it. Consequently:

- Any aggregate $\sum_k P(r_{i_k}) \cdot f(c_k)$ with growing f (data-movement distance uses $f = \sqrt{}$) is asymptotically governed by the highest-scale levels — exactly the levels whose miss-ratio relevance at any fixed cache size is zero.
- Conversely, the miss ratio at a fixed cache size is a *truncation* of the distribution that ignores those levels entirely.

So the distribution has two conjugate readings — probability-weighted (what misses at cache C) and value-weighted (how much data exists) — and no scalar represents both. A scalar’s leading term additionally erases the plateau constants, which is where realistic-cache behavior lives. The object of interest is the function $mr(C, n)$, and the symbolic model’s distinctive property is that this function has a *finite exact representation*.

Empirical confirmation (tables/dmd_inversion.md; suite at $n = 2016$, DMD/access = $\sum w \cdot \sqrt{rd}/A$, the scalar aggregate at that size):

- Order inversion: **gesummv** has DMD/access 33.8 vs **mvt** 29.4 — the scalar says gesummv moves data “farther” — yet gesummv misses 3.3x less at 32 KB (0.047 vs 0.156) and the two are equal at 1 MB.
- Near-ties that differ 4-6x in real traffic: doitgen/3mm/2mm (DMD ~ 34.3 -35) vs gesummv (33.8) differ 6.0x in miss ratio at 32 KB; gemver (24.5) vs gemm / lu_decomp / floyd_warshall (24.4-24.5) differ 3.6-3.7x.
- Growth exponents compress: kernels with 9x different cache behavior at 32 KB (0.031 to 0.28) all have per-access DMD exponents in a narrow band 0.83-0.95. The exponent is close to useless as a discriminator at machine scales.

None of this says the aggregate is wrong as what it is — one moment of the distribution, meaningful for distance/energy questions. It says the aggregate is not a locality *ranking*, and its leading term is not a locality *model*.

3. The regime reading

The proposal is to treat the finite level list as the primary output of the analysis, and every practical question as one of four functionals. gemm (PolyBench version, all bounds n , block 8; tables/suite_regimes.md) serves as the worked example. Its displayed levels:

boundary (lines)	portion	miss ratio after	physical scale
7/8 ... 45/8	31/32 total	1/16	lines and accumulators
$n/4$	1/32	1/32	two matrix rows
$n^2/8$	1/32	1/(16n)	one $n \times n$ matrix
$3n^2/8$	1/(16n)	0	all data (imaginary level)

The four functionals, each answered in closed form:

1. **Prediction:** $mr(C, n)$ at a machine point — evaluate the staircase. At $n = 2016$: $mr = 0.0312$ at 32 KB *and* at 1 MB (the $n/4$ -to- $n^2/8$ plateau spans 32 KB to 31 MB — three cache levels are equivalent for this kernel), dropping to $3.1e-5$ at 32 MB.
2. **Problem-size planning** (inversion in n): the one-matrix regime is active iff $n^2/8 + 3n/8 - 2 \leq C$, i.e. $n^*(C) \approx \sqrt{8C}$: $n \leq 64$ at 32 KB, $n \leq 362$ at 1 MB, $n \leq 2048$ at 32 MB. No profiling sweep produces this inverse; the polynomial does.
3. **Provisioning** (inversion in C): smallest cache with $mr \leq \tau$. For $\tau = 1/25$ across the suite see tables/machine_map.md — e.g. gemm 32 KB, 2mm/3mm 142 KB (their $9n/8$ -line boundary at $n = 2016$), trmm 472 KB, correlation 414 KB.

4. **Transformation accounting:** a program transformation acts on the diagram — it moves plateaus, moves boundaries, or inserts levels — and the symbolic difference is the benefit, as a function of C (Section 6).

The plateau constants — $1/16$, $1/32$, $3/8$ — are precisely the content the leading-term reading discards, and they are what distinguishes machines: at every cache from 32 KB to 16 MB, the whole story of gemm vs its variants is in those constants.

4. Anchors and self-checks

4.1 Reproduction of the paper’s tables (tables/anchors.md)

- **Table 1** (element granularity): reproduced *exactly*, including the fractional averages $c(ri) = 2n + 2 - 1/n$ and $n^2 + 3n - 1/n$ and all portions and plateaus. The two imaginary rows (RIs $3n^3 - 3n^2 + 3n$ and $3n^3 - 3n + 3$) merge into one displayed level at scale $3n^2$, which is the intended regime granularity; the fine split is retained internally.
- **Table 6** (block 8, min-max co-scaling): the boundary structure reproduces — constant boundaries, a $9n/8$ -line boundary, an $n^2/8$ -line boundary; plateaus constant, constant, $\Theta(1/n)$, 0. Small constant offsets are consistent with the paper’s array padding (its Section 4.1), which these runs do not apply.

4.2 RI Sum Invariance as a diagnostic

Evaluated on raw distributions at $n = 8400$:

kernel	sum ri·P(ri)	data size D	gap
matmul3 (block 1)	2.11672e8	2.1168e8	4.0e-5
matmul3 (block 8)	2.64348e7	2.6460e7	9.5e-4
gemm (block 8)	1.76326e7	2.6460e7	0.33

The first two confirm the identity to the precision of the analyzer’s degenerate-region filtering. The gemm gap is exactly one matrix ($n^2/8$ blocks): the filter dropped one array’s cross-run imaginary region, and the invariant *localizes* the loss. The only effect is on the final sliver of the staircase (the D-level plateau reads $1/(16n)$ instead of $\sim 3/(32n)$); all constant-scale and row-scale statements are unaffected. This is the intended use of the invariance: not a proof of correctness, but an audit that tells you what is missing and whether it matters for the question asked.

4.3 Mass conservation and excluded kernels

Summed rd mass must equal the access count (infinite repeat). 22 of 28 suite kernels conserve to within 0.5–0.2%. Six do not, in either execution model, and are excluded from every table (tables/summary.json):

kernel	best coverage
heat-3d	1.544
jacobi-1d	1.331
jacobi-2d	1.273
seidel-2d	0.929
fdtd	0.959
imperfect	0.969

The over-counts (jacobi, heat-3d) are scale-independent — they persist when the time-step parameter is bound to $n/21$ — so they are an analyzer inconsistency in the symbolic RD decomposition of time-stepped stencils at symbolic bounds, not a modeling choice. The point for the framework: **the algebra is self-checking**. A numeric simulator that double-counted 13% of a trace would silently produce a plausible curve; a distribution that violates conservation by 4/3 is caught mechanically.

Convolution is a genuinely two-scale kernel (image n , filter f with $f \ll n$); binding both to n makes every chamber empty. A 9x9-filter variant conserves at 0.9999 and behaves as expected (mr 0.0077 at 32 KB, 0.0015 at 1 MB). Multi-scale kernels need multi-scale sections — which the polynomials support directly; only the one-parameter presentation needed the binding.

4.4 A caveat the checks cannot cover

The triangular kernels (cholesky, lu, syr, syr2k, symm) conserve mass but report implausibly small miss ratios at 32 KB ($4e-5$ to $7e-5$) and degenerate signatures (cholesky: constant boundary then straight to D). Conservation validates masses, not rd *values*; the scale approximation is known to under-resolve the largest reuse distances of triangular loops. Statements about this class are withheld until an exact-RD mode is available. (The regime machinery itself is indifferent to how rd is computed.)

5. The suite through the regime lens

Coarse signatures — for each distinct boundary scale, expressed as a power of the footprint D, the final plateau after it (tables/signatures.md):

class	signature	kernels
A	$O(1) \rightarrow \text{const}; \mathbf{D}^{1/2} \rightarrow \text{const}; D \rightarrow 0$	gemm, 2mm, 3mm, atax, bicg, mvt, gemver, gesummv, trmm, convolution9
B	$O(1) \rightarrow \text{const}; D^{1/2} \rightarrow \Theta(1/n); D \rightarrow 0$	syr, syr2k, symm, lu (<i>suspect, see 4.4</i>)
C	$O(1) \rightarrow \text{const}; D^{1/3} \rightarrow \text{const}; D^{2/3} \rightarrow \text{decay}; D \rightarrow 0$	correlation, covariance, doitgen, gramschmidt
C'	$O(1) \rightarrow \text{const}; D^{1/4} \rightarrow \text{const}; D^{1/2} \rightarrow \text{decay}; D \rightarrow 0$	floyd_warshall, lu_decomp
E	$O(1) \rightarrow \text{const}; \mathbf{D} \rightarrow \mathbf{0}$ (no intermediate boundary)	trisolve

Readings:

- Class A is the classical shape: a row-scale boundary at $\Theta(\sqrt{D})$ drops the miss ratio to a lower constant, and only the full footprint ends it. Within a class, kernels differ *only in constants* — which is exactly what the leading-term view cannot see and the machine map shows: at 32 KB, class A spans 0.031 (gemm) to 0.50 (trmm, correlation) to 0.67 (gramschmidt, class C).
- Class C/C' kernels have multiple intermediate scales; their diagrams have two knees below the footprint, at $D^{1/3} / D^{2/3}$ (deg-3 footprint) or $D^{1/4} / D^{1/2}$ (deg-4).
- trisolve is the counterexample shape: nothing between the row cluster and the whole matrix. Its miss ratio is pinned at 1/32 until the cache holds all data.

6. Transformations move the diagram

(tables/tiling.md; 3-access matmul family, block 8, infinite repeat, $N = 2048$.)

Loop interchange moves a plateau and no boundary. Same body, ijk vs ikj order:

variant	4 KB	32 KB	512 KB	64 MB
ijk (k inner)	0.374	0.374	0.0415	4.1e-5
ikj (j inner)	0.0831	0.0831	0.0417	4.1e-5

A 4.5x traffic difference at every cache below 512 KB, converging above. Both variants have identical boundary structure and identical asymptotics; the entire effect is a plateau constant (3/8 vs 1/12: the k-inner order walks B by column and pays a full line per element). This is the common case of real tuning, and it is invisible to any order-of-growth analysis.

Tiling inserts a boundary at the tile scale and lowers plateaus.

variant	4 KB	8 KB	32 KB	512 KB	16 MB	64 MB
naive	0.374	0.374	0.374	0.0415	0.0415	4.1e-5
tile 8	0.0104	0.0104	0.0104	0.00525	0.00525	4.1e-5
tile 16	0.00781	0.00749	0.00523	0.00523	0.00264	4.1e-5
tile 32	0.0443	0.0443	0.00353	0.00262	0.00134	4.1e-5

Pointwise traffic gains over naive run 36x to 106x at practical sizes, and 1.0x at 64 MB — tiling changes plateaus, not the footprint boundary, so all variants converge once a matrix fits. Two regime effects worth stating precisely:

- **Tile choice is a regime question.** tile 32 is the best variant at 32 KB (its 3-tile working set is 24 KB) and the *worst* at 8 KB and below (0.0443 vs naive-beating 0.0104 for tile 8). The crossover cache sizes are the tiled kernels’ own boundaries — the diagram ranks tile sizes per cache level with no simulation.
- **The gain law has constants.** In the tile-resident regime the plateau is $\Theta(1/b)$ with the analyzer’s exact coefficients, so the best achievable gain at cache C scales as $\sqrt{C}$ with a known constant — the pointwise counterpart of the tiling headroom, valid at every C rather than only asymptotically.

7. Side study: co-scaling and scaling laws

The min-max scaling of the paper (its Section 4.8) generalizes across the suite: to *hold* regime k while the problem grows, the cache must grow like $c_k(n)$ — a specific power of the data size with a specific coefficient. The signature table of Section 5 is therefore the suite’s scaling-law table:

- The $\sqrt{2}$ rule (cache $\propto \sqrt{\text{data}}$ preserves the hit ratio) is exactly the statement that the active boundary has scale $D^{1/2}$. It holds for class A kernels *in the window between the row boundary and the footprint boundary* — e.g. for gemm from 32 KB to $n^2/8$ lines — with known plateau; it fails below (constant regime: any cache works) and above (footprint regime: cache must be $\propto D$). trisolve has no $D^{1/2}$ window at all; class C kernels obey cube-root rules instead (boundaries at $D^{1/3}$ and $D^{2/3}$). The rule of thumb becomes a theorem with a validity interval per kernel.
- Fixed machine, growing problem: as n grows past each $n_k^*(C)$, the miss ratio steps up through the plateau list. The sequence of critical sizes $n_k^*(C) = c_k^{-1}(C)$ is the kernel’s “cache-cliff schedule”, closed form. For gemm at 1 MB the last cliff is at $n = 362$.

- Can data-movement complexity express scaling laws? As a single moment of the measure, it scales with the *largest* boundary only, so it reproduces the footprint-regime law and nothing else; the empirical exponent compression of Section 2 is the same fact seen numerically. The signature — the full exponent *list* with coefficients — is the scaling law; the scalar is its last entry.

8. Side study: parallel extrapolation by parameter substitution

The cache polynomials are symbolic in every loop bound separately. If p workers each execute a contiguous slice of one loop, the per-worker access stream *is* the original kernel with that bound divided by p — so the per-worker staircase requires no new analysis, only substitution. This models private caches and ignores interleaving and coherence; its value is that boundary structure under decomposition becomes readable. (tables/parallel.md; gemm, $n = 2016$.)

Aggregate traffic = $p \times \text{worker accesses} \times \text{worker mr}$, private caches:

- **i-slicing (rows of the output): parallelism cannot help locality.** The re-swept matrix ($k \times j$) stays whole in every worker; every boundary is invariant under the substitution $p0 \rightarrow n/p$. Measured: aggregate traffic $1.02e9$ lines at 32 KB *and* at 1 MB, for every p from 1 to 1008 — flat. p caches of 1 MB never act as more than one.
- **j-slicing (columns): aggregate capacity works, with a threshold.** Each worker re-sweeps an $n \times n/p$ slice; the whole-matrix boundary becomes $\sim n^2/(8p) + \Theta(n)$ lines. At 1 MB the model predicts residency at $p \approx 31$; measured: traffic collapses 61x exactly at $p = 32$ ($1.02e9 \rightarrow 1.68e7$ lines), then grows again $\propto p$ (each worker still pays the $\Theta(n)$ -line row terms — the additive constants in the boundary matter, and the diagram has them).
- **k-slicing** behaves like j-slicing shifted one step (residency at $p = 63$, not 32, because the sliced panel keeps whole n -element rows — again the additive terms), plus visible extra accesses from the unsliced output updates.
- **A line-granularity floor.** At 32 KB, j-slicing *never* reaches residency: a sliced matrix still spans at least n lines (each of n rows touches ≥ 1 line), and $n = 2016$ lines = 126 KB > 32 KB. Worse, once the slice width drops below the 8-element block, spatial locality degrades and the measured per-worker miss ratio *rises* (0.030 at $p = 32$ to 0.141 at $p = 1008$). Element-level reasoning about parallel working sets misses both effects; the block-granular polynomials contain them.

The GPU reading of these facts is direct but stated here as a program, not a result: a thread block's shared-memory tile is a cache of size C executing a substituted sub-kernel, so the tiling table of Section 6 evaluated at $C = 48\text{--}228$ KB *is* the shared-memory regime analysis; and whether cross-block reuse survives in L2 is the same invariant-vs-shrinking-boundary question as i- vs j-slicing, at $C_{\text{effective}} = L2/(\text{active blocks})$. What is missing for a quantitative GPU model is a validated interleaving rule for concurrent streams (shared-footprint theory is the natural bridge) — the symbolic per-worker curves are the input to that rule, and they are already free.

9. Limitations

- The scale approximation under-resolves triangular kernels' largest reuse distances (Section 4.4); regime statements for class B are withheld. An exact-RD mode would slot into the same pipeline.
- The analyzer's symbolic RD decomposition for time-stepped stencils violates mass conservation at symbolic bounds (Section 4.3); those six kernels await an analyzer-side fix. The conservation check should run wherever the analyzer runs.
- Multi-scale kernels (convolution) need explicit scale bindings for a one-parameter presentation; the polynomials themselves are fine.

- Array padding (used in the paper’s evaluation) is not modeled here; it shifts constant terms of block-granular boundaries (Section 4.1).
- Everything targets a single-level fully associative LRU abstraction, as in the paper; the parallel section is a first-order model without interleaving or coherence.

10. What this buys the paper

Existing tools answer locality questions in one of two currencies. Simulation and profiling produce *numbers*: one configuration at a time, no inversion, cost proportional to the sweep. Asymptotic analysis (I/O complexity, data-movement complexity) produces *exponents*: no constants, no boundaries, blind to everything that distinguishes 32 KB from 16 MB on a real machine. The algebraic model is the only analysis that produces the object in between — and this study’s claim is that the object, not any scalar collapsed from it, is the product:

A loop nest’s locality is a finite list of regimes: polynomial cache-size boundaries with rational miss-ratio plateaus between them. The list is small (four entries for gemm), exact, carries its own consistency checks, and is symbolic in every program parameter at once. Every question practitioners actually ask — will this problem fit; what cache does it need; which variant, tile size, or parallel decomposition wins at *this* cache level and by how much; how must cache co-scale with data — is a single polynomial evaluation or inversion on the list. The vanishing-probability imaginary levels, far from being negligible, are what carry the footprint boundary and cross-invocation reuse: discarding them (or collapsing the list to its leading term) is precisely what makes every scalar summary misrank real kernels at real cache sizes.

That is a motivation that does not lean on prediction accuracy (already established), does not inherit the fragility of complexity-style aggregates (Section 2 shows they invert), and gives the compiler a product with uses beyond prediction: regime tables as documentation, transformation accounting, provisioning formulas, scaling laws with validity windows, and substitution-based parallel reasoning.

Files

path	contents
qp.py	exact piecewise-quasi-polynomial parser/evaluator, polynomial fitting, region-domain satisfiability (Fourier–Motzkin)
regimes.py	level extraction: exact fits, scale clustering, portions/plateaus; writes regimes/<kernel>.<model>.json
derived.py	staircase evaluation at concrete sizes (guard-correct), displayed levels, signatures, machine map, DMD comparison, tiling tables
parallel_study.py	slicing-by-substitution study (gemm)
anchor_checks.py	Table 1 / Table 6 reproduction, RI Sum Invariance
run_suite.py	regenerates data/ from dsl/ with dmd-cli (block 8, scale, both models)

path	contents
dsl/	analyzer inputs: extracted PolyBench kernels (sym_*), the matmul family (matmul3*, written for this study), 9x9 convolution
data/	raw dmd-cli JSON (not committed if large; regenerable)
regimes/, tables/	extracted structure and all tables cited above

Reproduce

```
python3 run_suite.py           # dmd-cli over dsl/ -> data/ (~10 min, 8-way)
python3 regimes.py             # exact level extraction -> regimes/
python3 derived.py             # suite tables -> tables/
python3 parallel_study.py      # gemm slicing table
python3 anchor_checks.py       # paper anchors + invariance
pandoc REPORT.md -o REPORT.pdf --pdf-engine=xelatex \
  -V mainfont="DejaVu Serif" -V monofont="DejaVu Sans Mono" \
  -V geometry:margin=1in
```